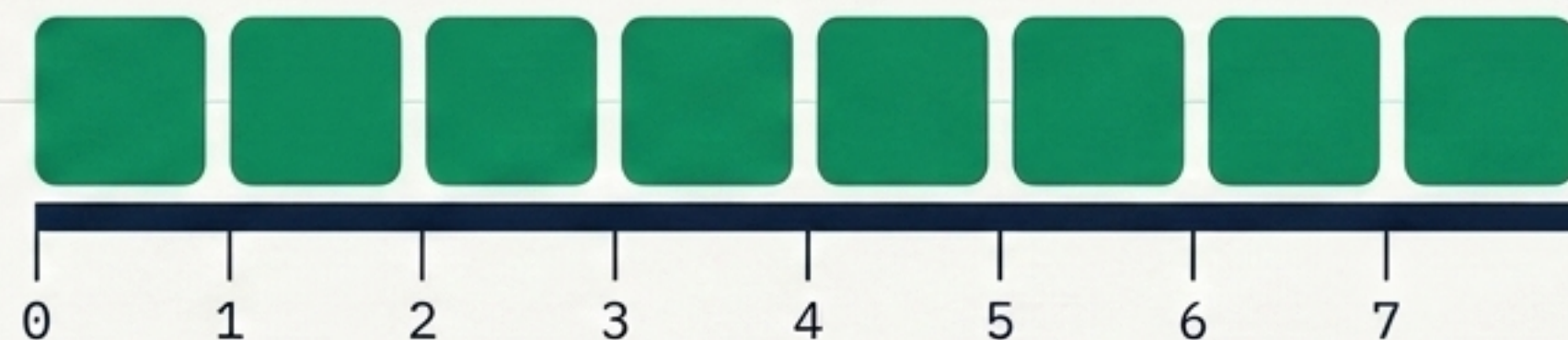
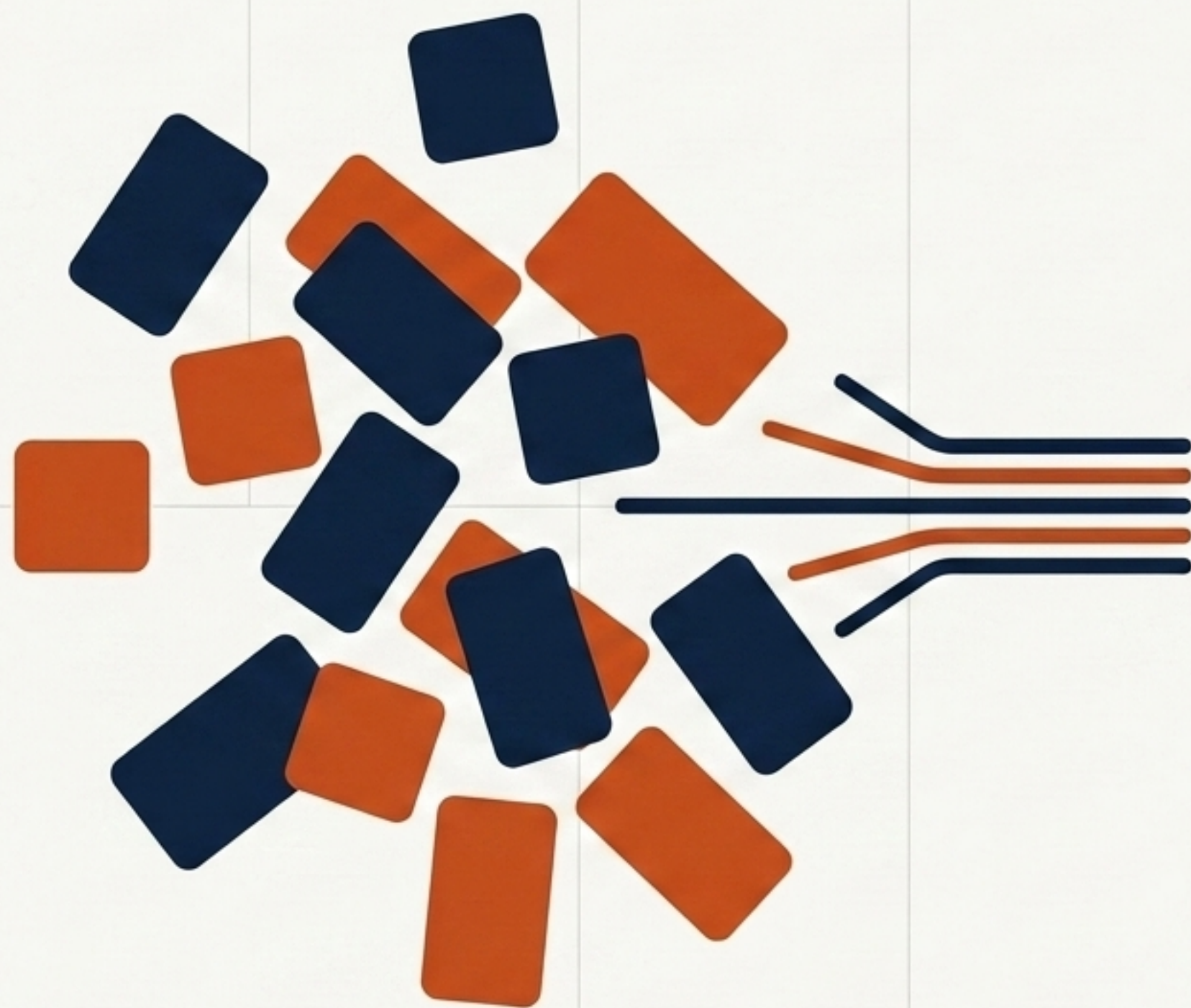


Optimización de Procesos: Ordenamiento de Datos y Complejidad Algorítmica

De la teoría $\theta(n \log n)$ a un caso práctico en contabilidad.



Orden de
Llegada



Fecha
Límite

El Problema Operativo:

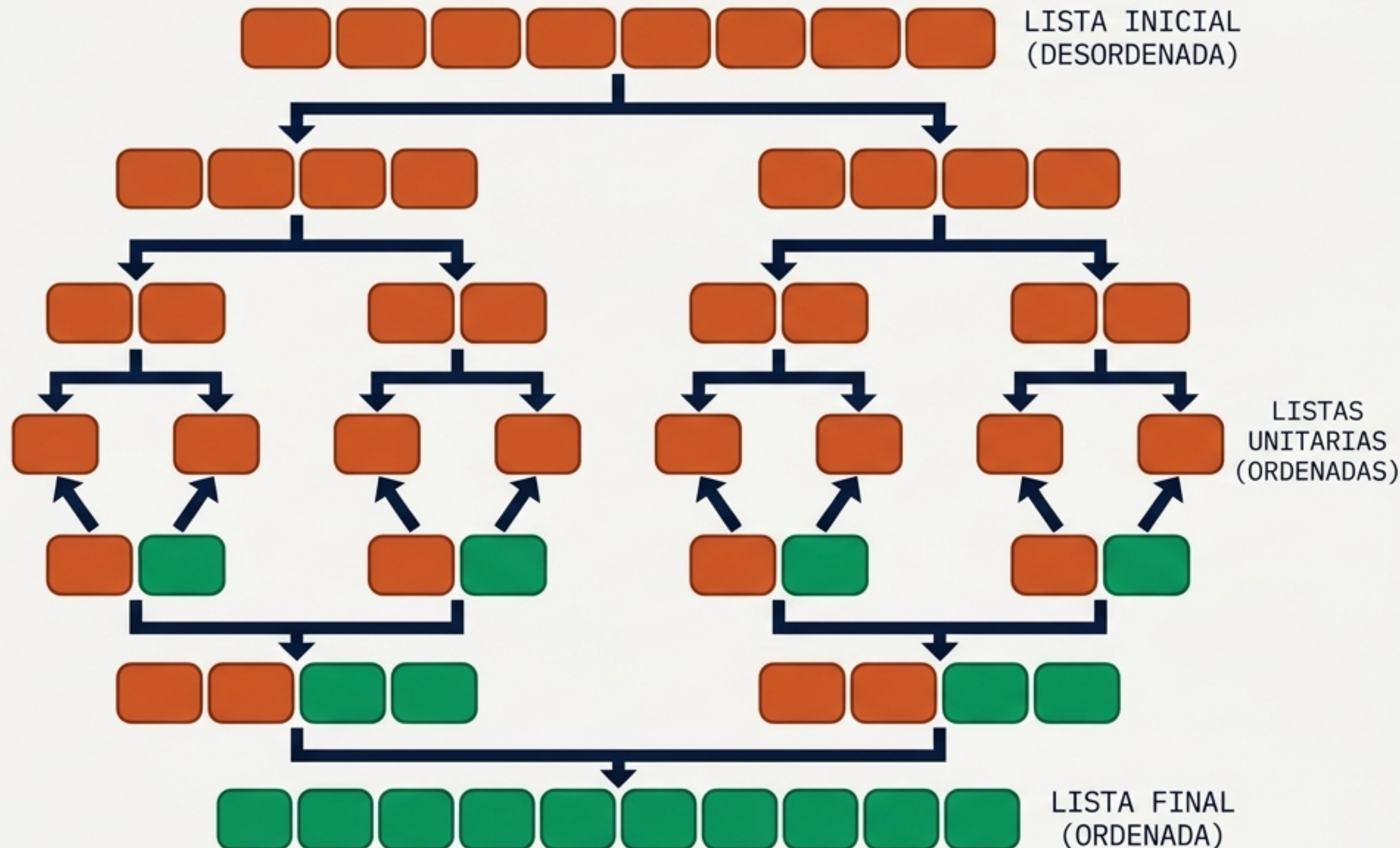
El contador recibe documentos sin un orden relacionado con la fecha de entrega.

La Restricción: El procesamiento debe priorizar estrictamente la fecha límite impuesta por el NIT.

El Requisito:

Un algoritmo robusto que garantice rendimiento sin importar el nivel de desorden inicial de los datos.

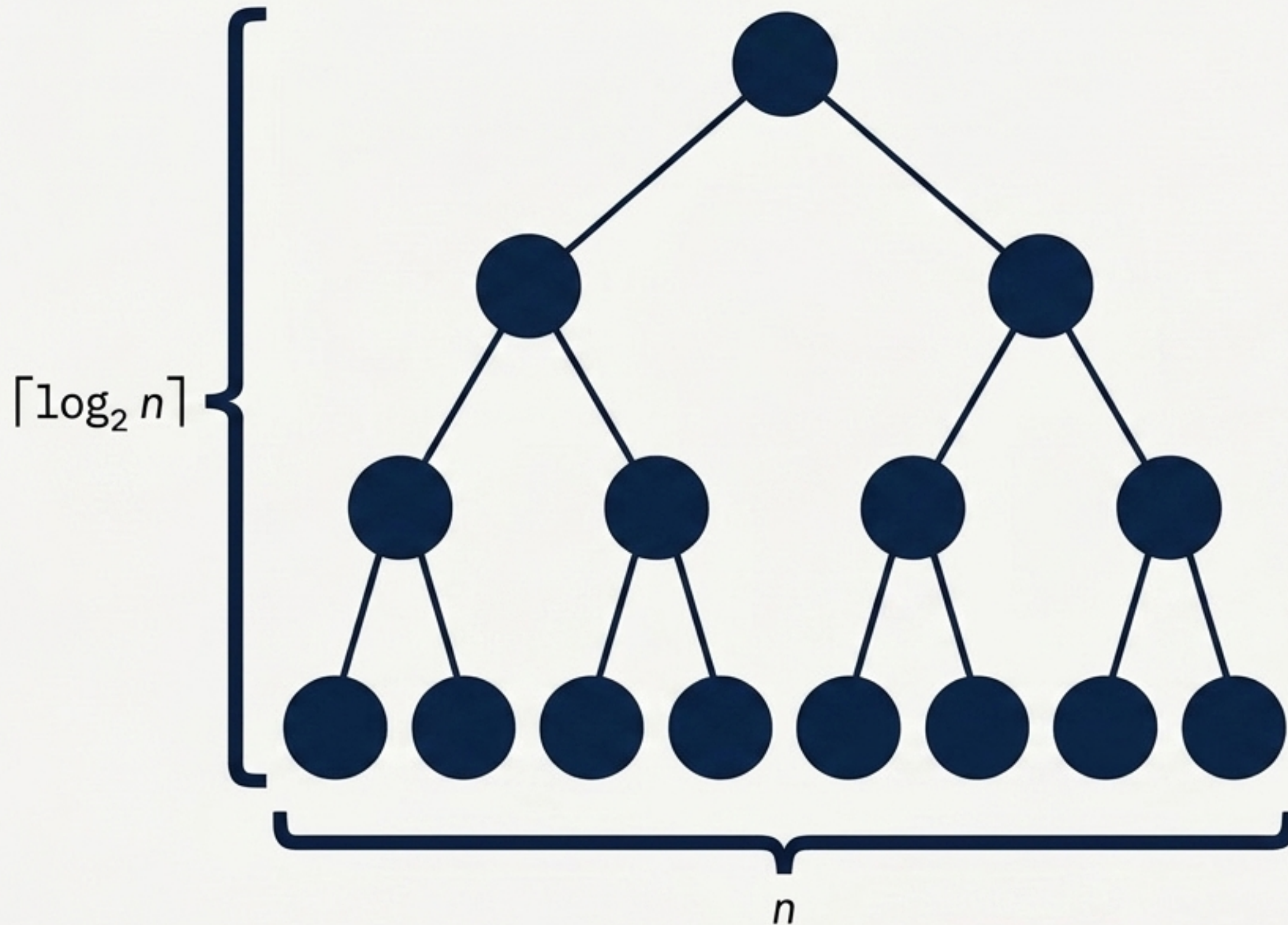
El Motor: Merge Sort (Divide y Vencerás)



1. Dividir: Ignora el caos inicial partiendo la lista recursivamente hasta obtener listas de 1 solo elemento (trivialmente ordenadas).

2. Fusionar: Combina las sublistas ordenadas en un arreglo final, comparando solo los frentes de cada sublista.

Anatomía de la Complejidad



Profundidad del Árbol:
Dividir a la mitad produce $\log_2 n$ niveles.

Costo por Nivel:
Fusionar en cada nivel requiere recorrer todos los n elementos $\rightarrow O(n)$.

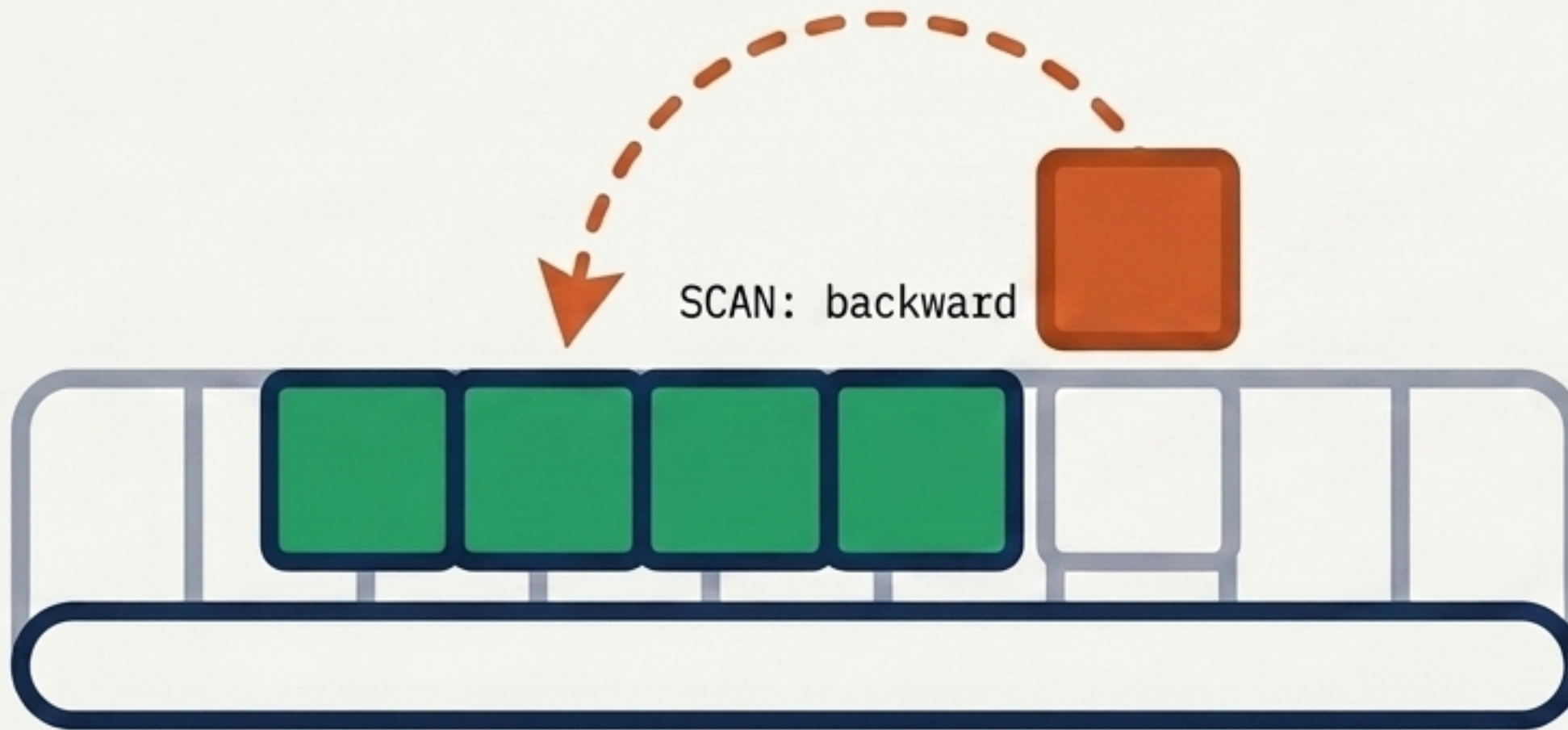
Rendimiento Garantizado:
El costo total es invariablemente $\Theta(n \log n)$ en el mejor, peor y caso promedio. Es la opción blindada para un sistema desde cero.

El Arma Secreta: Estabilidad



1. **El Desempate Implícito:** Muchos clientes comparten la misma fecha límite (mismos dígitos de NIT). ¿Quién se procesa primero?
2. **Conservación del Orden:** Un algoritmo estable respeta el orden original de entrada (`orden_llegada`) en caso de empate.
3. **La Clave en el Código:** `if izquierda[i].fecha_limite <= derecha[j].fecha_limite:`
(El operador `<=` prioriza la izquierda, preservando la historia).

El Contraste: Insertion Sort



La Mecánica:

Recorre la lista, toma el elemento actual y lo desplaza hacia atrás hasta su posición correcta en el segmento ya ordenado.

El Superpoder (Adaptabilidad):

Si el contador procesa un lote casi ordenado (clientes entregando justo a tiempo), los desplazamientos son mínimos y el costo colapsa a $\theta(n)$.

El Riesgo:

En caos total (peor caso), el costo se dispara a $\theta(n^2)$.

Matriz de Diagnóstico: Merge vs. Insertion

Algoritmo	Complejidad (Peor)	Memoria	Adaptativo	Estable	Uso Ideal
Merge Sort	$\Theta(n \log n)$	$\Theta(n)$ Extra	No	Sí	Lotes masivos y caóticos desde cero.
Insertion Sort	$\Theta(n^2)$	$O(1)$ In-place	Sí (Mejor: $\Theta(n)$)	Sí	Insertar un cliente rezagado en una cola ya procesada.

Tarea 2: Más Allá del Ordenamiento Clásico



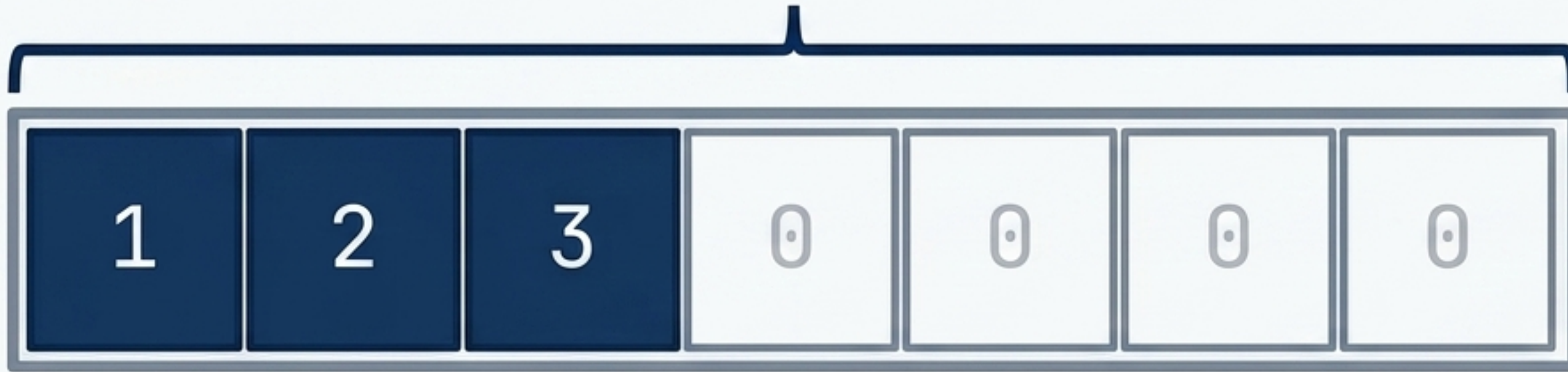
La Barrera Teórica: Ningún algoritmo por comparaciones puede superar $\Omega(n \log n)$ en el peor caso.

Rompiendo las Reglas: En LeetCode, explotaremos restricciones específicas (datos ya parcializados o universos de claves minúsculos) para lograr tiempos lineales $O(n)$.

LeetCode 88: Merge Sorted Array

nums1

$m + n$



nums2

n



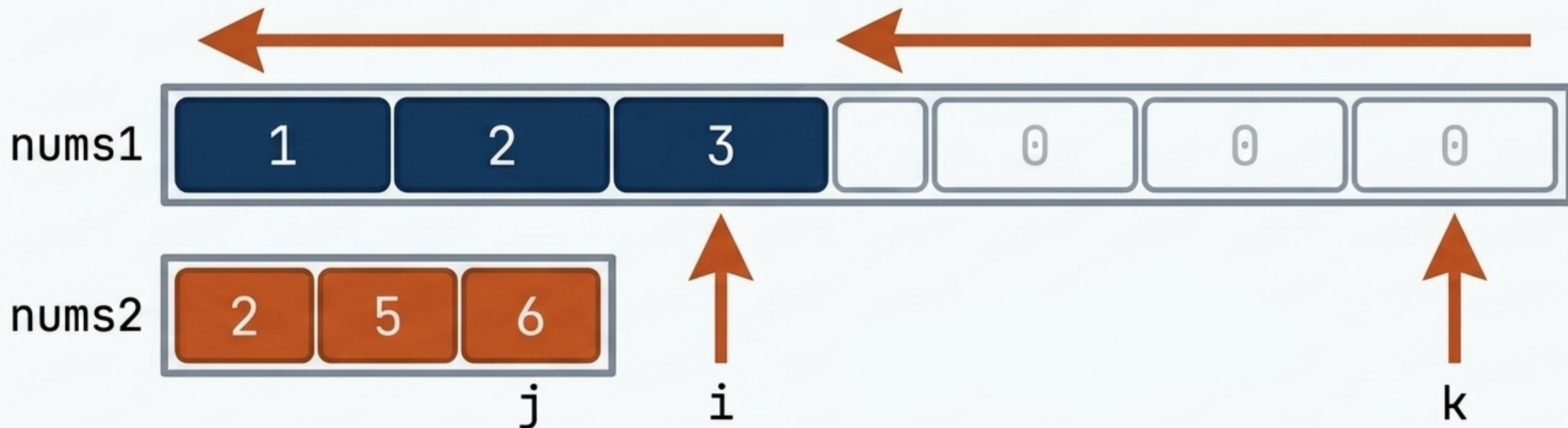
El Escenario:

Tenemos dos corridas que ya están ordenadas (idéntico al paso final de Merge Sort).

El Reto:

Fusionarlas in-place dentro de `nums1` sin sobrescribir los datos originales de `nums1` que aún no han sido leídos.

La Solución: Fusión Inversa (Two Pointers)



La Mecánica:

Escribir desde el final hacia el principio ($k = m + n - 1$). El mayor de `nums1[i]` y `nums2[j]` ocupa el espacio final de `nums1`.

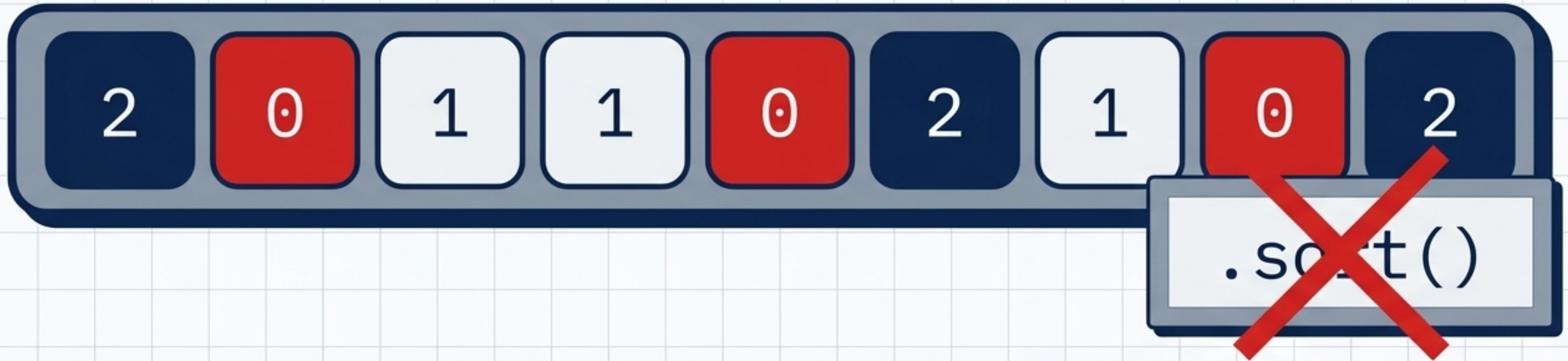
Por qué funciona:

Al escribir de atrás hacia adelante, garantizamos que el espacio vacío absorbe los datos sin pisar valores no leídos.

Complejidad:

Tiempo $O(m + n)$,
Espacio extra $O(1)$.

LeetCode 75: Sort Colors

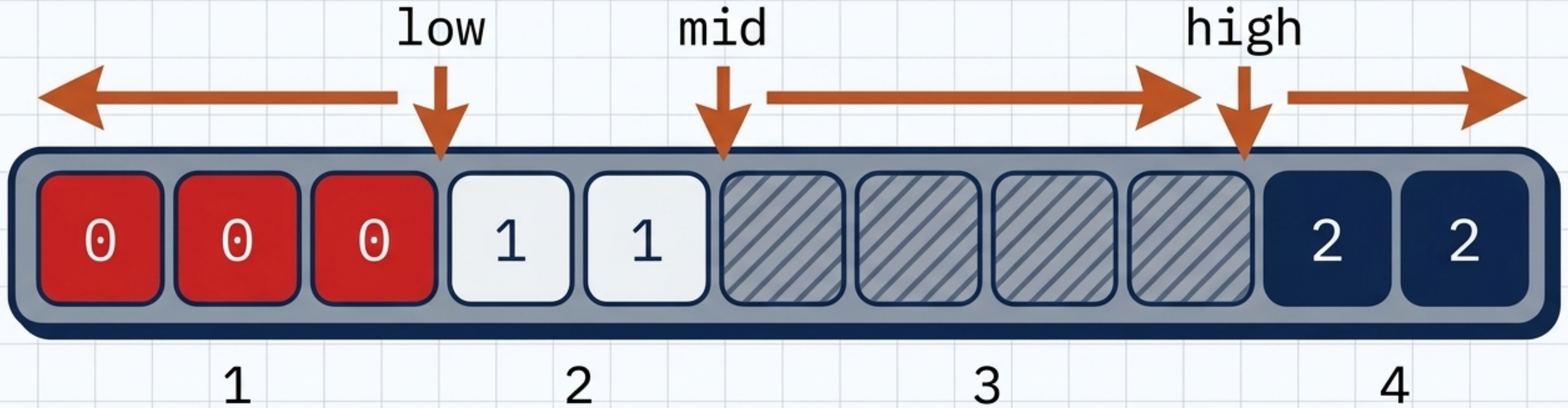


El Escenario: Un arreglo masivo compuesto únicamente por tres valores posibles: 0, 1 y 2.

El Reto: Ordenar el arreglo in-place agrupando los colores.

La Restricción: Está prohibido usar la función nativa de ordenamiento de la librería.
Se exige una sola pasada ($O(n)$).

Rompiendo la Barrera: Tres Punteros



La Estrategia (Bandera Holandesa):

Un solo recorrido. Dependiendo del valor apuntado por `mid`, el elemento se envía a la frontera de los 0s o a la frontera de los 2s.

¿Por qué es $O(n)$?:

Como el universo de claves es minúsculo ($k=3$), no necesitamos ordenar por comparaciones exhaustivas. Solo ruteamos el tráfico.

Complejidad:

Tiempo $O(n)$,
Espacio $O(1)$.

Síntesis Final: El Algoritmo Dicta la Política

Caos total con empates (DIAN) — **Merge Sort** — $\Theta(n \log n)$

Lote casi ordenado — **Insertion Sort** — $\Theta(n)$

Fusión de ya ordenados (LC 88) — **Two Pointers** — $O(m+n)$

Universo de 3 claves (LC 75) — **Bandera Holandesa** — $O(n)$

Conclusión: La elección del algoritmo óptimo depende de las restricciones y características específicas del problema y los datos de entrada.